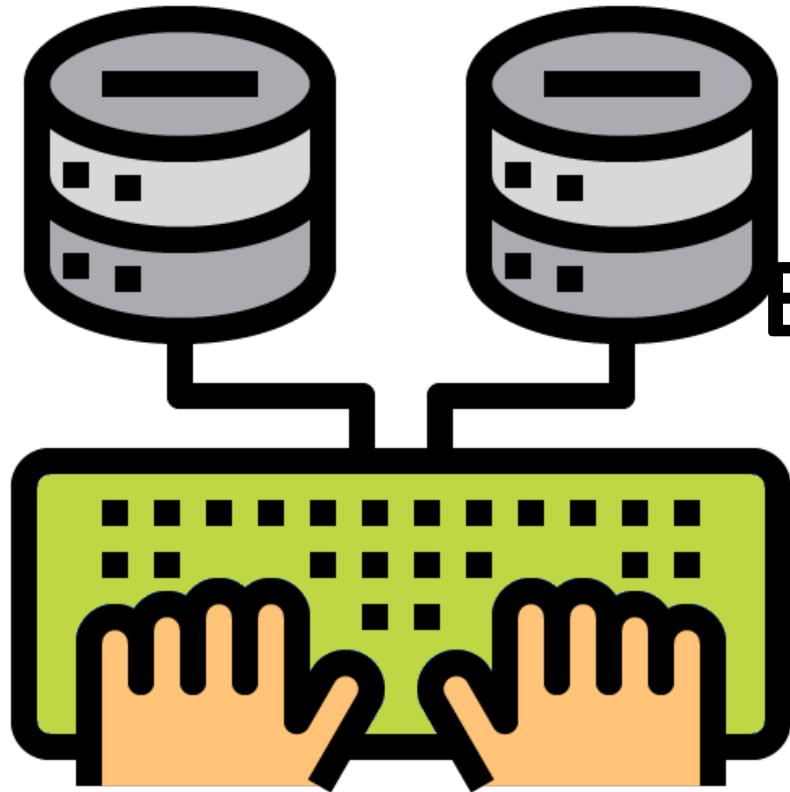


Big Data Analytics

Distributed Processing Basic Concepts and Principles



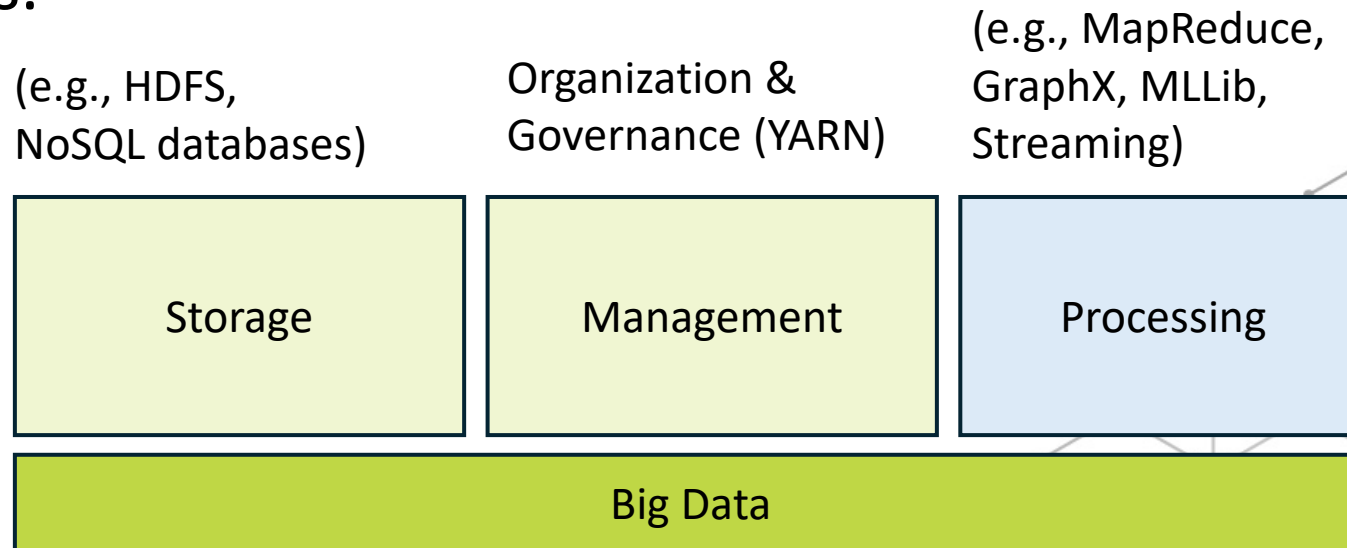
Lecturer:

Márcia Lourenço Baptista
m.baptista@novaims.unl.pt

Distributed Processing

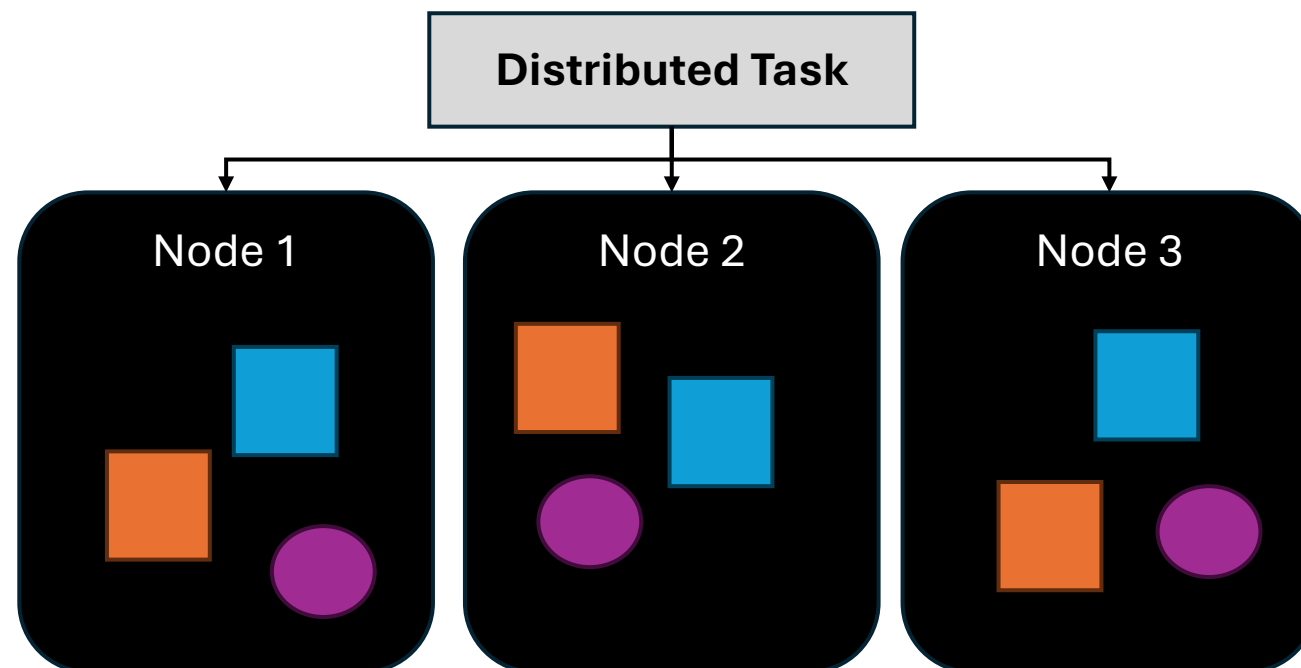
Distributed systems refers to a variety of systems that use more than one physical computer or node to run an application.

Big data processing is the techniques or programming models to access large-scale data to extract useful information for making decisions.



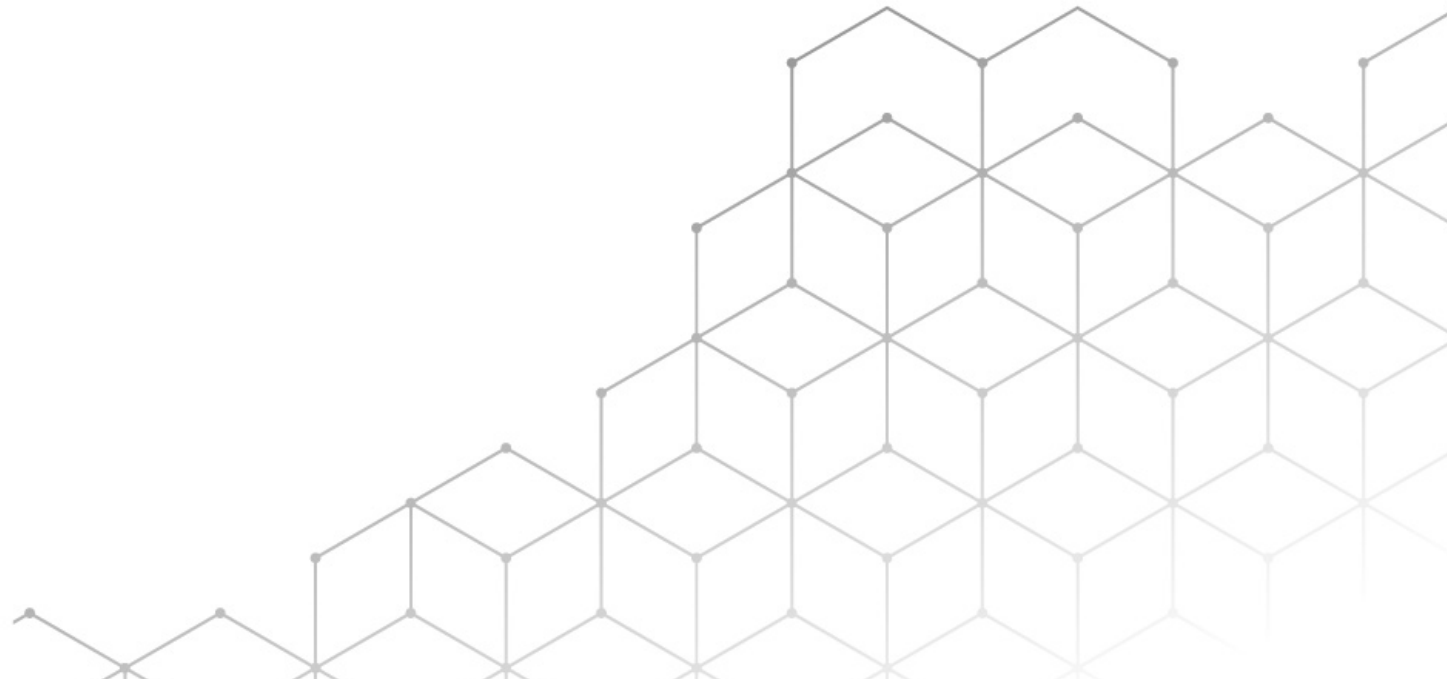
Big Challenge of Big Data Processing

- Need an understanding of the data and the need to maintain data integrity (if the data set is broken into pieces)
- Need to synchronize parallel processing.



Learning Objectives

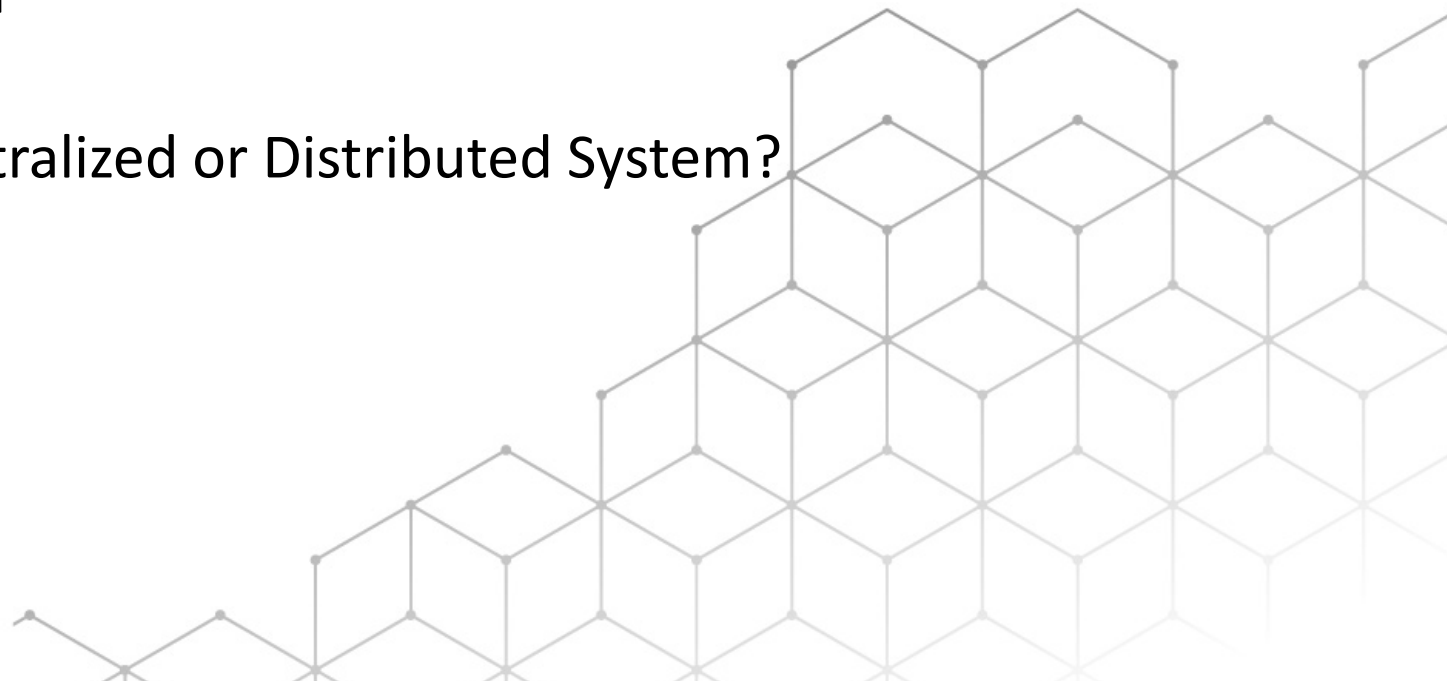
- Today we have the following agenda:
 - Distributed Processing
 - Data Processing Techniques
 - Share nothing vs Share everything



Agenda for Today

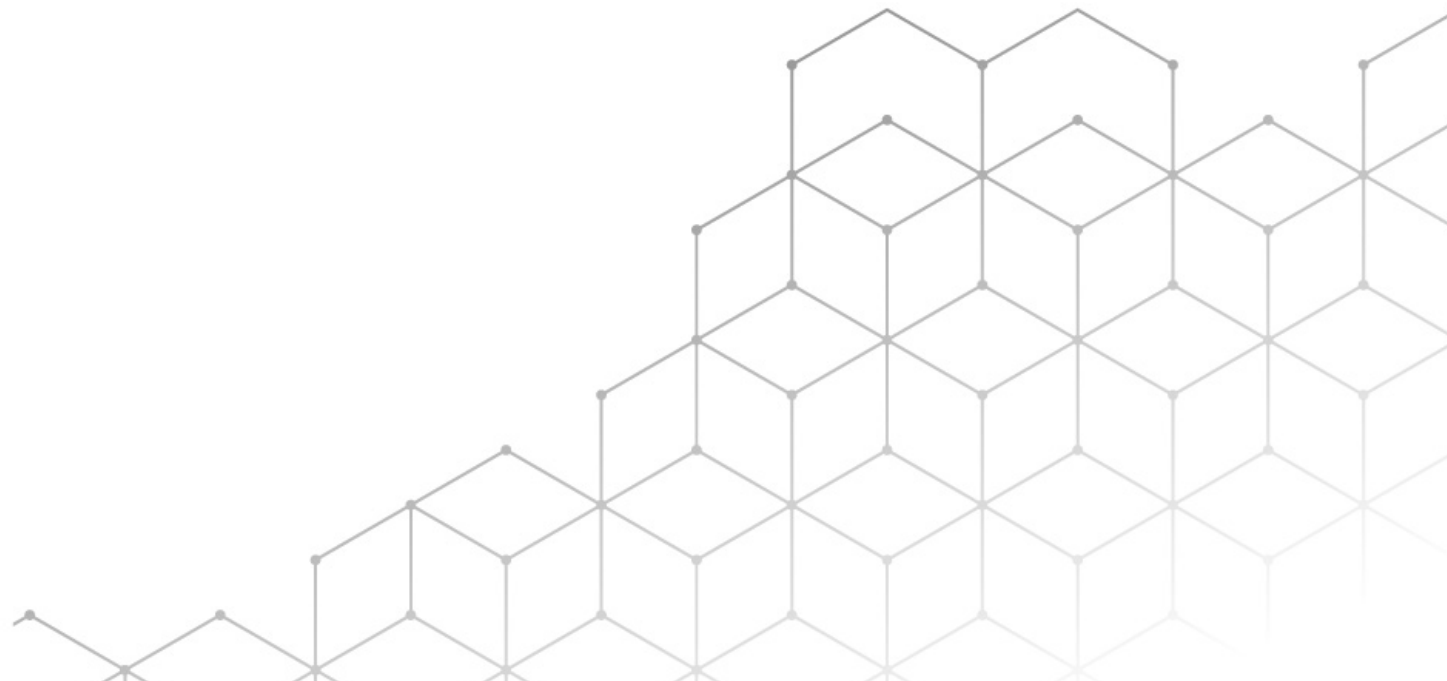


- Distributed Processing Architectures: What are the roles of nodes?
 - Data Processing Techniques
 - Centralizes Processing System
 - Decentralized Processing System
 - Distributed Processing System
 - Question and Answers
 - Is Hadoop Centralized, Decentralized or Distributed System?
- Infrastructure Architecture



Data Processing Techniques

- **Centralized processing:** all computation is done at a central server
- **Decentralized processing:** computation is done by central servers
- **Distributed processing:** computation is distributed among nodes



Centralized Processing System

Definition: A single central authority controls the entire system.

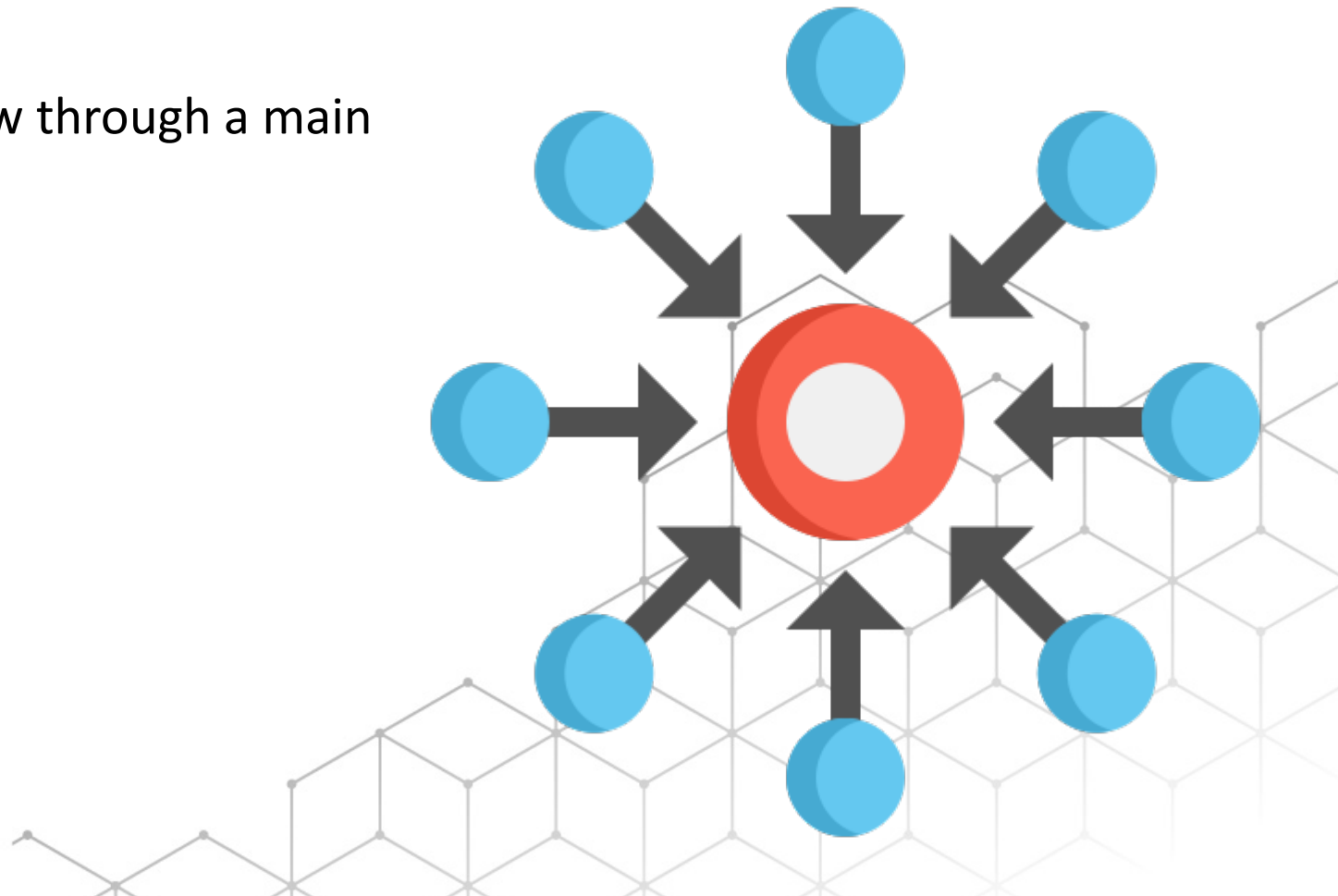
How it works: All data or decisions flow through a main node or server.

Pros:

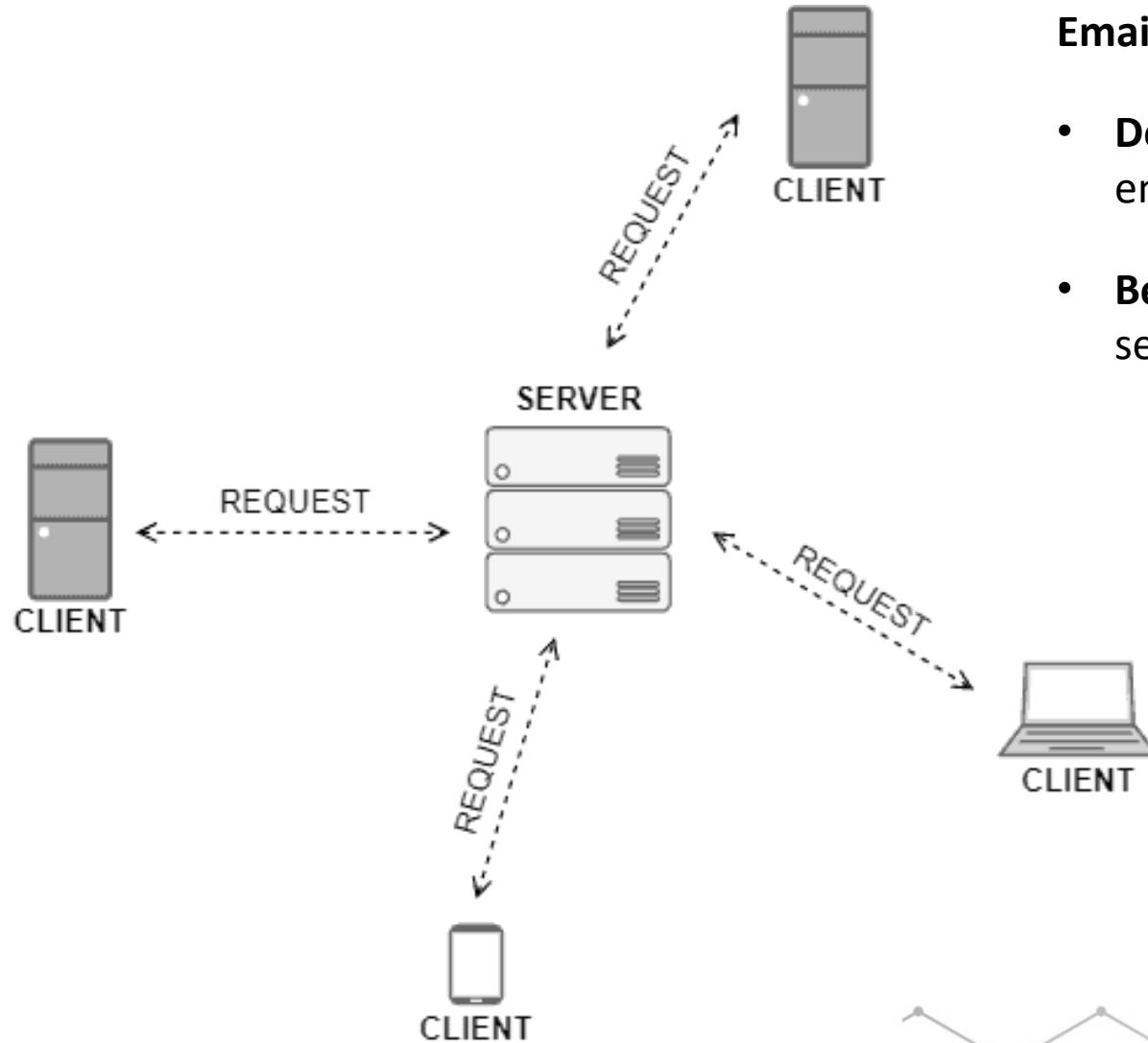
- Easier to manage and maintain.
- Simpler security model.

Cons:

- Single point of failure.
- Limited scalability.



Example of Centralized System



Email Server

- **Description:** Centralized email servers manage and store email communications for an organization.
- **Benefits:** Centralized email storage, simplified backup and security measures, and easy management of user accounts.

Decentralized Processing

Definition: Multiple authorities or nodes control different parts of the system, but not everything is connected to one center.

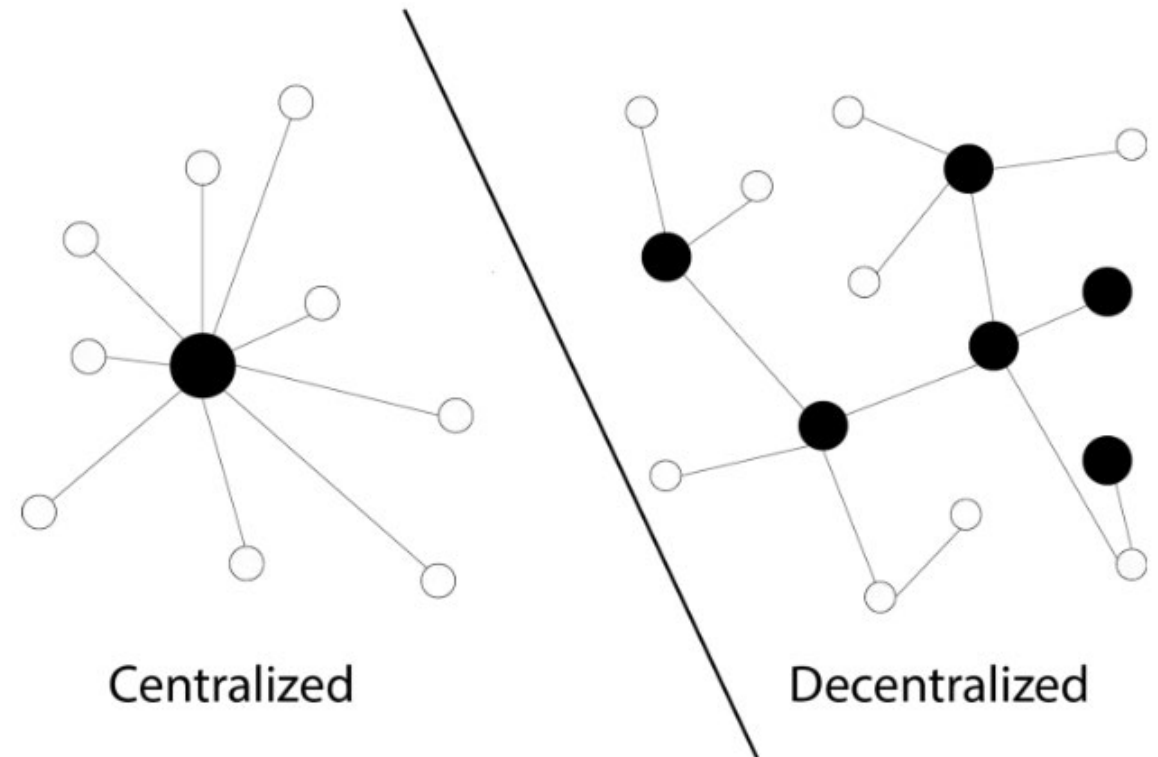
How it works: Each node makes decisions independently or collaborates with others, but there's no central hub.

Pros:

- More resilient (no single point of failure).
- More scalable than centralized systems.

Cons:

- Coordination can be harder.
- Requires trust or consensus mechanisms.



Distributed System

Definition: The system is spread across multiple nodes that work together, appearing as a single system to users.

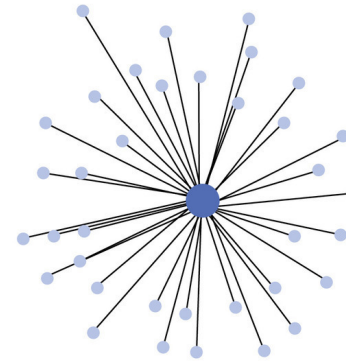
How it works: Tasks, data, and control are shared across all nodes.

Pros:

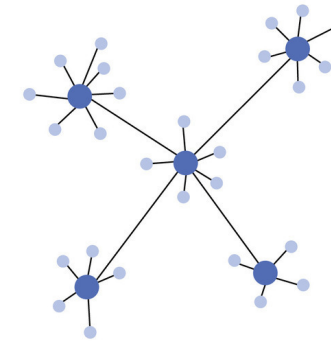
- High fault tolerance.
- Efficient resource usage.

Cons:

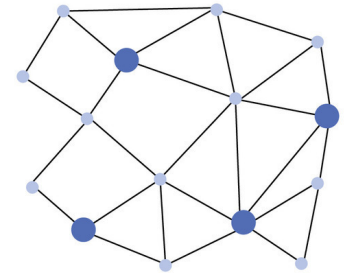
- Complex design and synchronization.
- Harder to debug or manage.



Centralized



Decentralized



Distributed

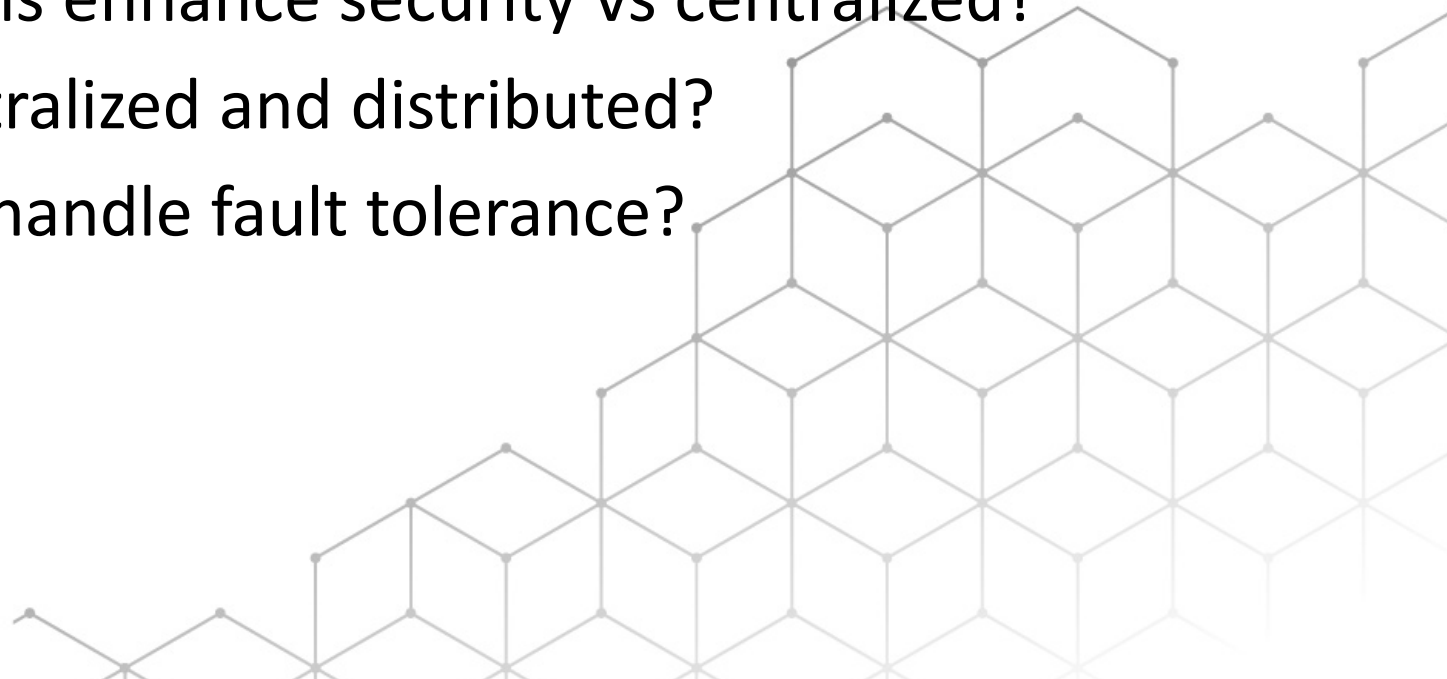
Questions

- Why might an organization choose a centralized system over a decentralized or distributed one?
- How do decentralized systems enhance security compared to centralized systems?
- Can a system be both decentralized and distributed?
- How do centralized systems handle fault tolerance compared to distributed systems?



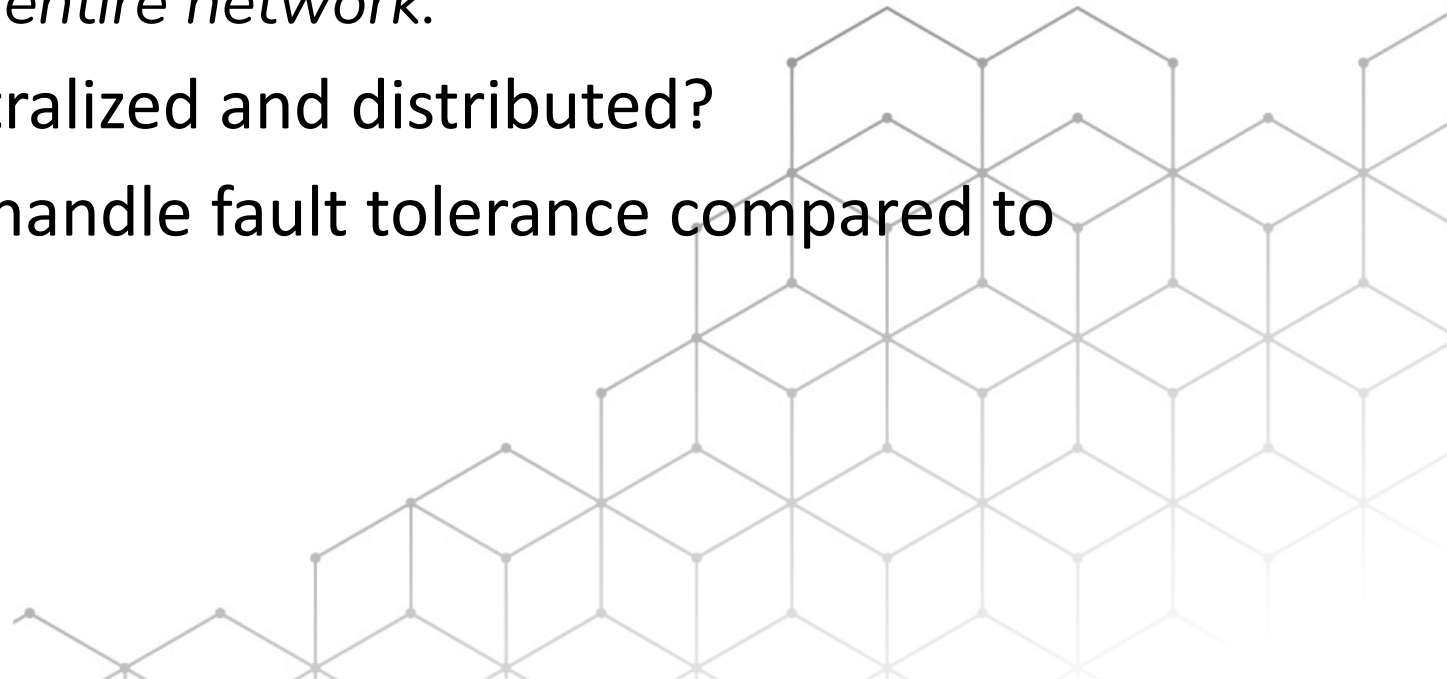
Questions

- Why an organization may choose a centralized system?
 - *Organizations might choose centralized systems for simplicity in management, easier implementation of security measures, and streamlined decision-making processes.*
- How do decentralized systems enhance security vs centralized?
- Can a system be both decentralized and distributed?
- How do centralized systems handle fault tolerance?



Questions

- Why an organization may choose a centralized system?
- How do decentralized enhance security vs centralized systems?
 - *By eliminating single points of failure and spreading control across multiple nodes. Each node operates independently, making it harder for attackers to disrupt the entire network.*
- Can a system be both decentralized and distributed?
- How do centralized systems handle fault tolerance compared to distributed systems?

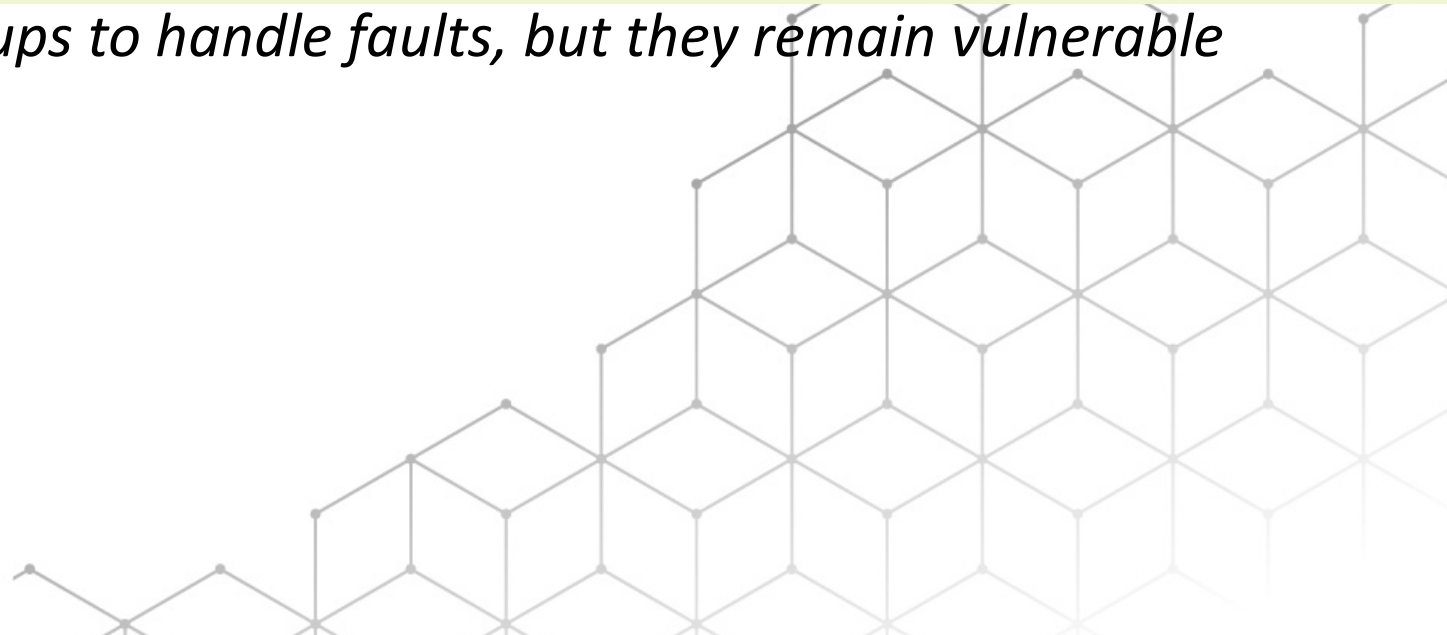


Questions

- Why an organization may choose a centralized system?
- How do decentralized systems enhance security vs centralized?
- Can a system be both decentralized and distributed?
 - *Yes, a system can be both decentralized and distributed. For example, blockchain networks are both decentralized (no single central authority) and distributed (data and operations spread across multiple nodes). These systems combine the benefits of both architectures, such as enhanced security and scalability.*
- How do centralized systems handle fault tolerance?

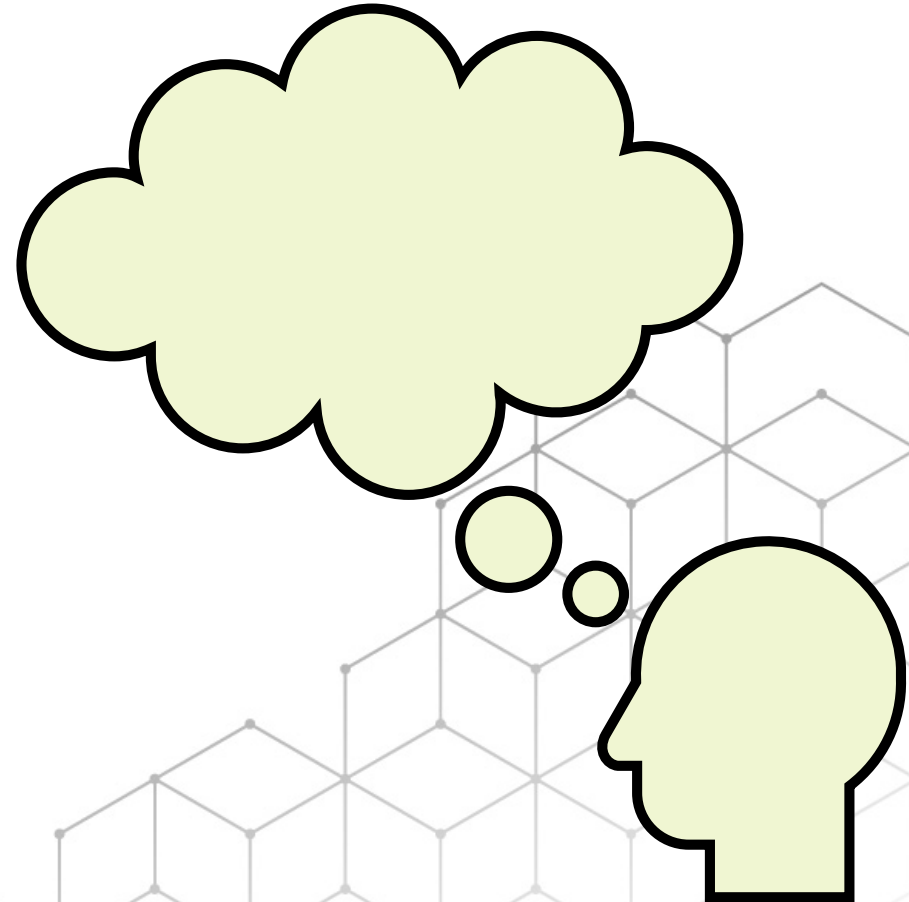
Questions

- Why an organization may choose a centralized system?
- How do decentralized systems enhance security vs centralized?
- Can a system be both decentralized and distributed?
- How do centralized systems handle fault tolerance?
 - *Rely on redundancy and backups to handle faults, but they remain vulnerable to single points of failure.*



Question

- So, is Hadoop **distributed**, **centralized**, or **decentralized**?



Hadoop is fundamentally distributed

Distributed: YES

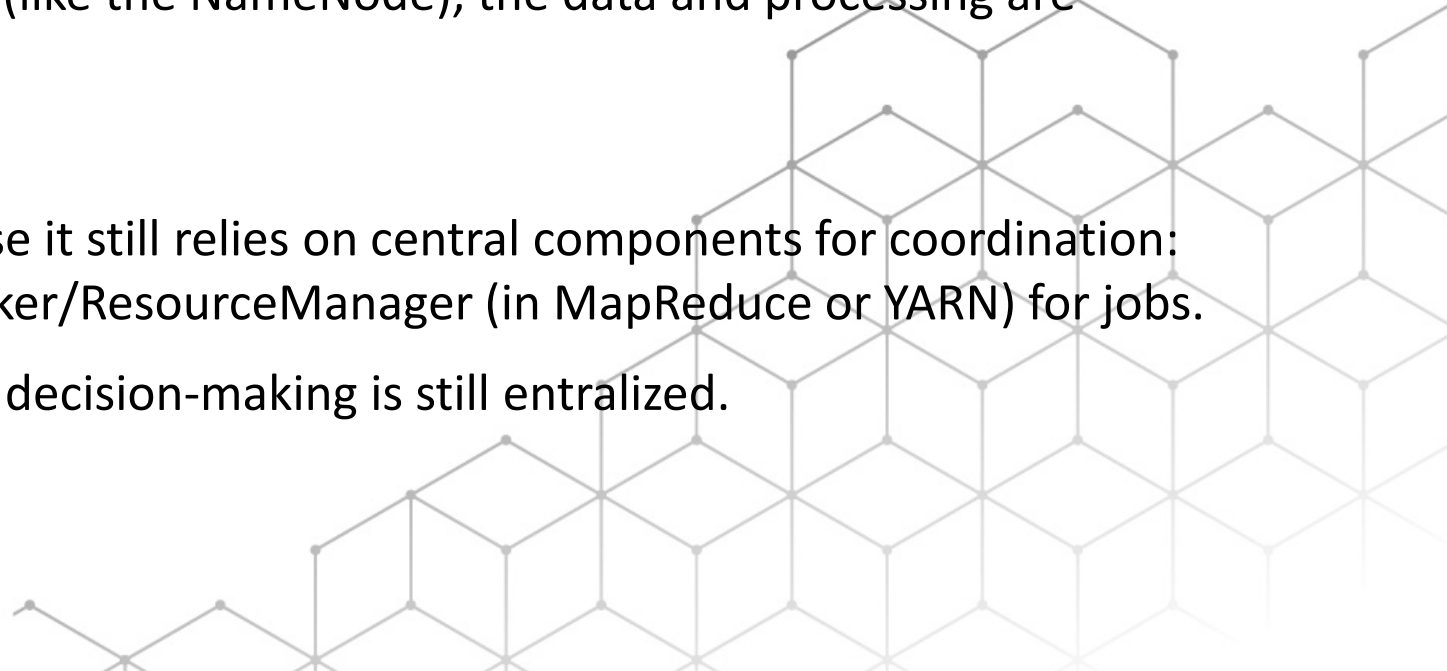
- Hadoop is a distributed system. Data and computation are spread across many machines (nodes). Each node stores a portion of the data and does local computation.

Centralized: Not exactly

- Even though it has a central component (like the NameNode), the data and processing are distributed.

Decentralized: Not exactly

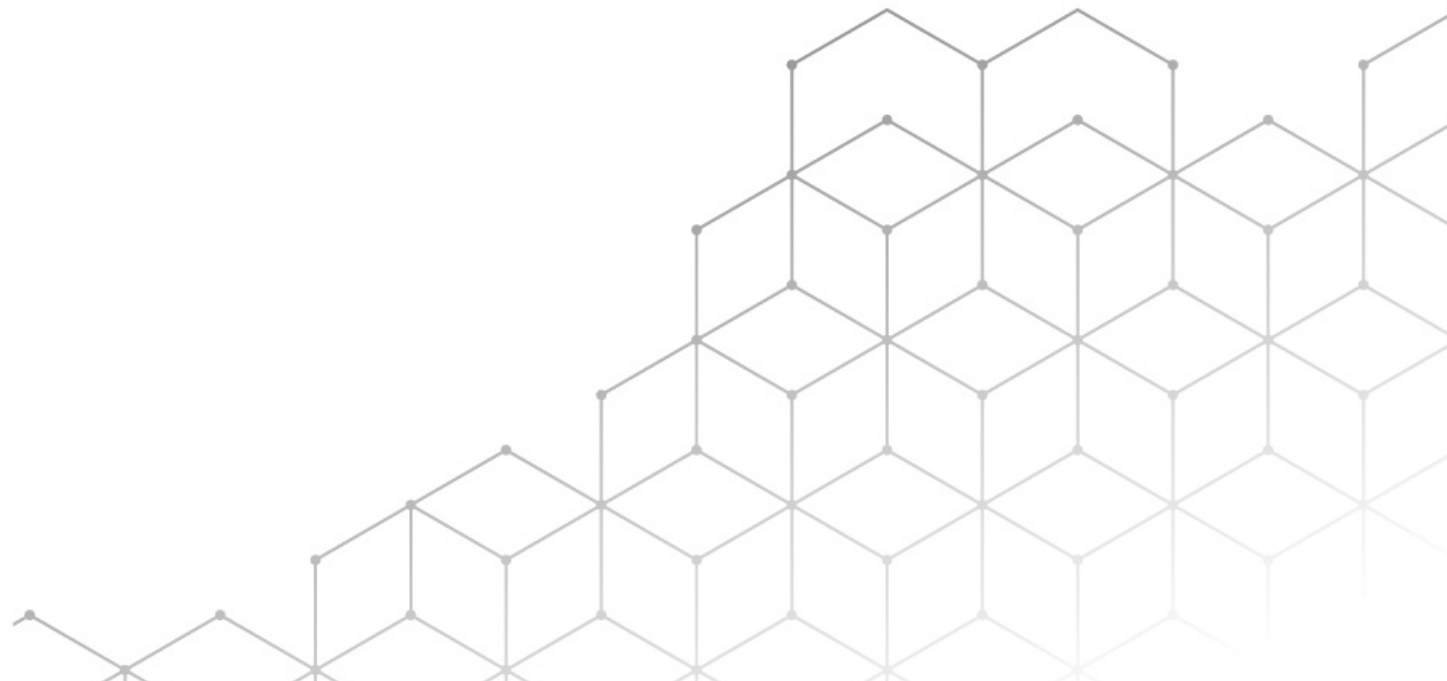
- Hadoop is not fully decentralized because it still relies on central components for coordination: NameNode manages metadata. JobTracker/ResourceManager (in MapReduce or YARN) for jobs.
- So, while data and tasks are distributed, decision-making is still centralized.



Agenda for Today

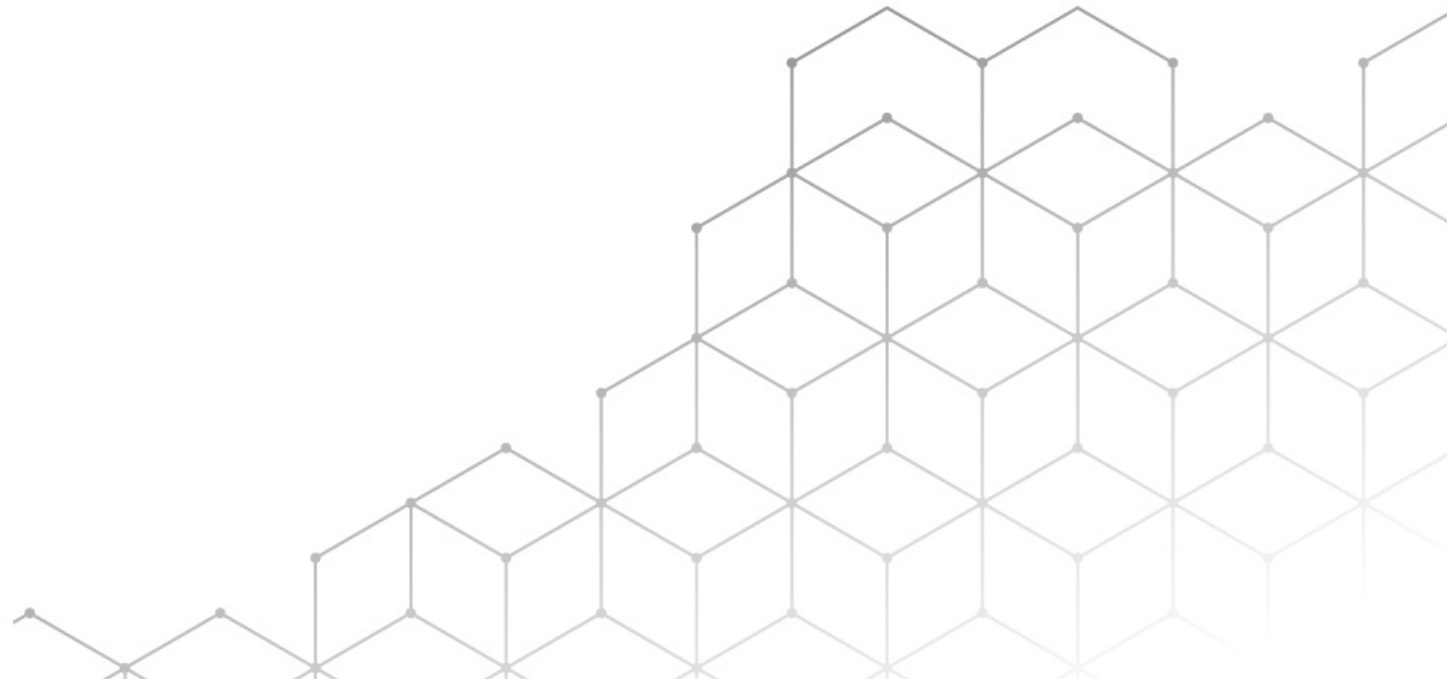


- Distributed Processing Architectures: What are the roles of nodes?
- Infrastructure Architecture: What are the resources of each node?
 - Share everything Architecture
 - Share nothing Architecture
 - Share memory vs Share disk
 - Share Hybrid



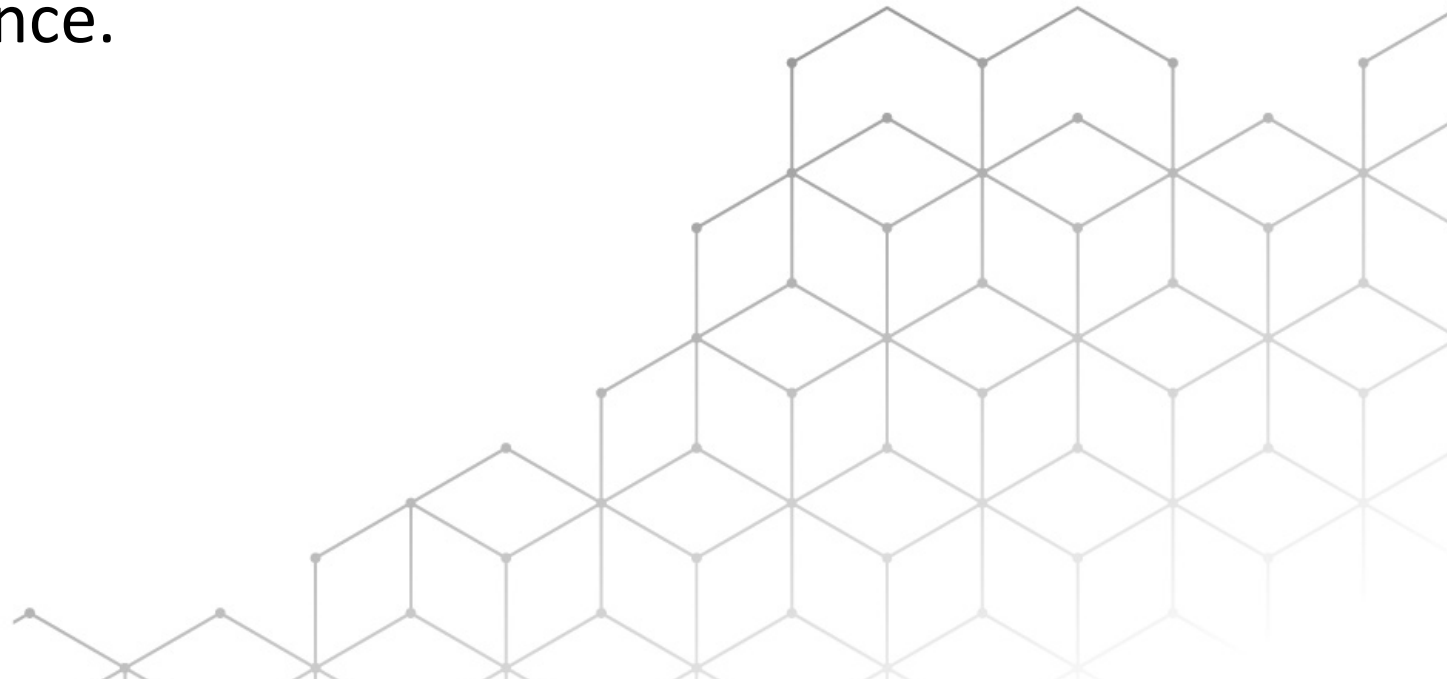
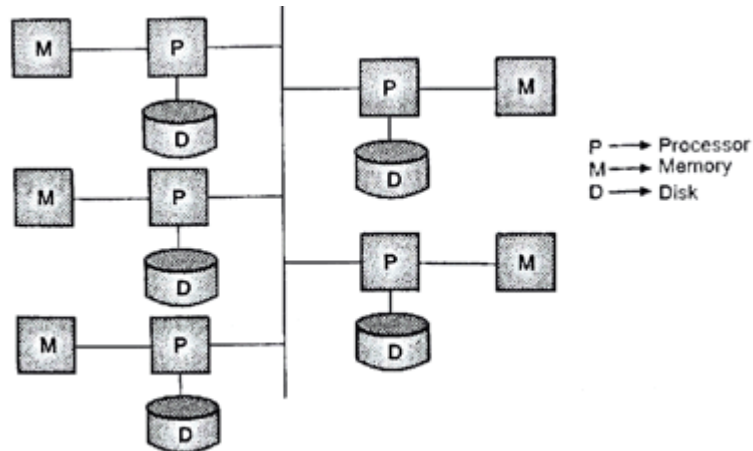
Shared Everything vs Nothing

- Shared everything architecture, all nodes have access to a common pool of resources, leading to potential contention and bottlenecks.
- In shared nothing architecture, each node has its own dedicated resources, eliminating contention.

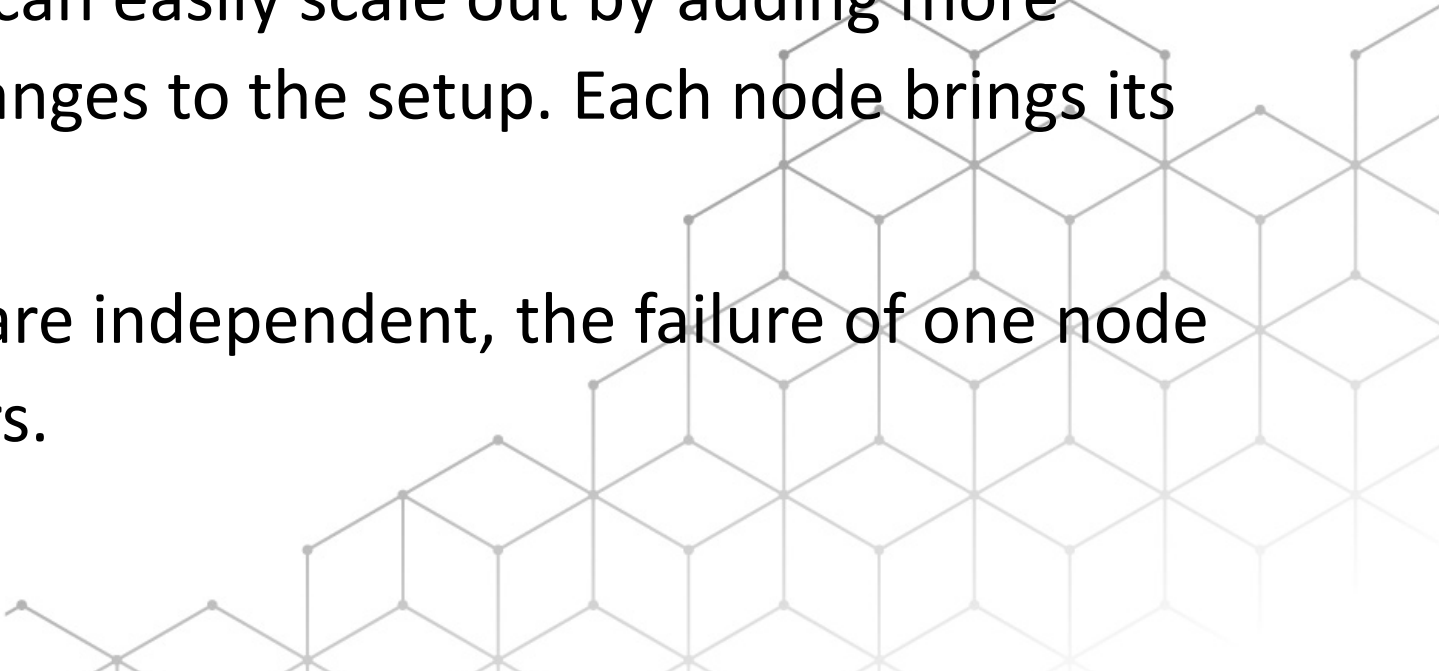


Architecture of Shared Nothing

- Processors share neither a memory nor disk. In this architecture, data accessed from local disks (and local memory accesses) do not pass through interconnection network, without resource sharing.
- Shared-nothing multiprocessors can be scaled up to thousands of processors without interference.

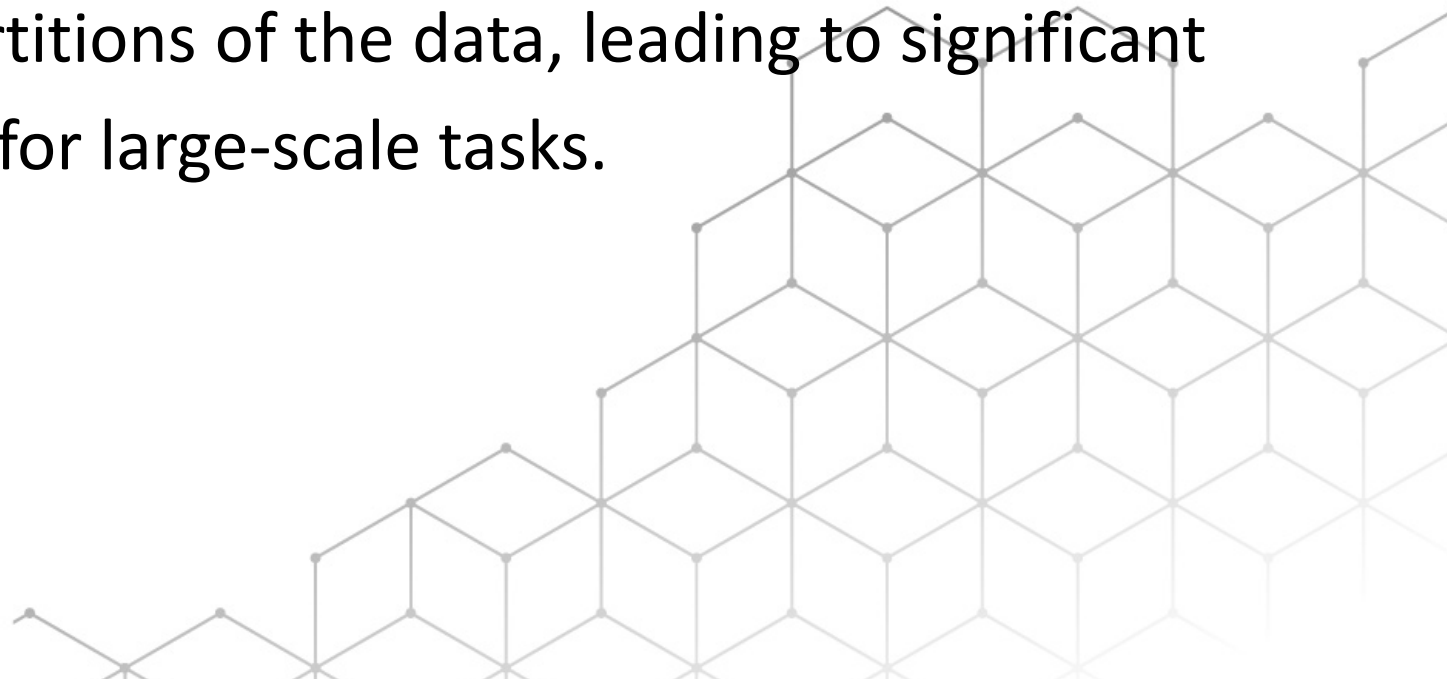


Characteristics of Shared Nothing (1/2)

- **Independence:** Each node operates independently and does not share disk storage or RAM with other nodes. Nodes communicate via network.
 - **Scalability:** The architecture can easily scale out by adding more nodes without significant changes to the setup. Each node brings its own memory and storage.
 - **Fault Isolation:** Since nodes are independent, the failure of one node does not directly affect others.
- 

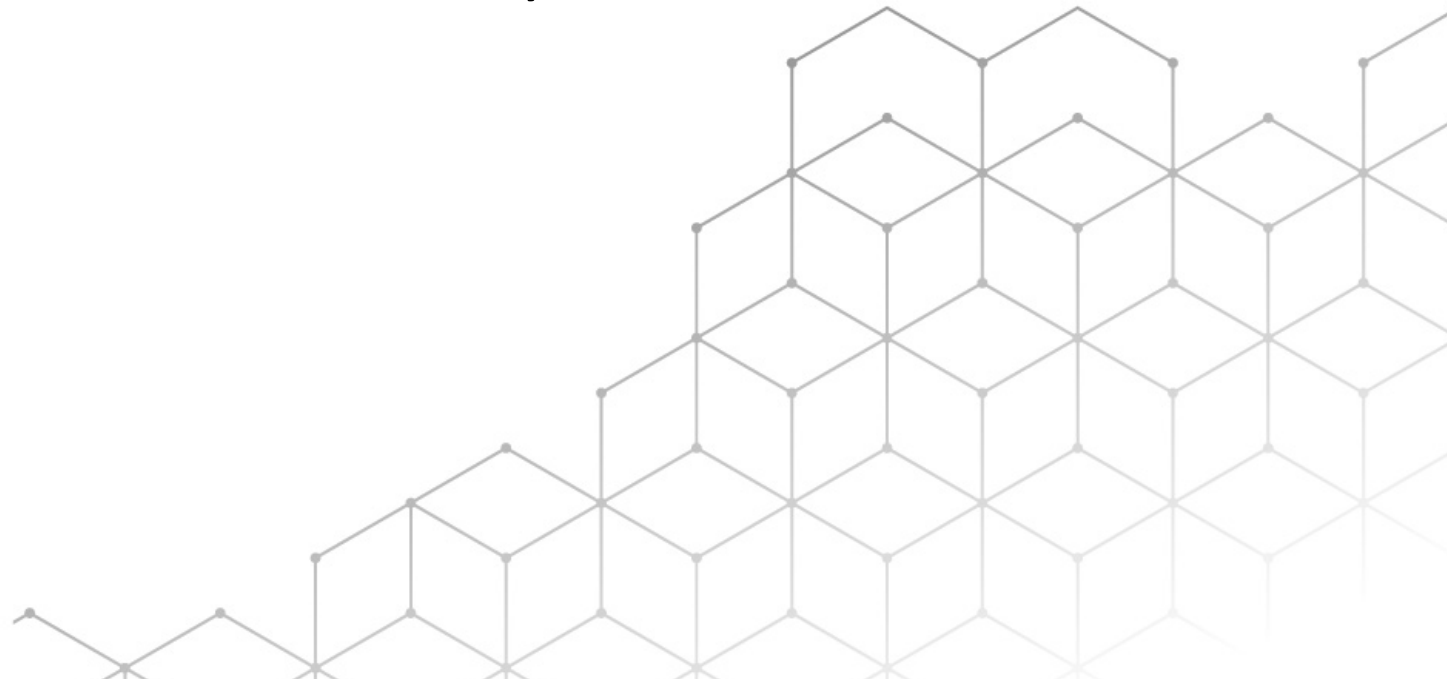
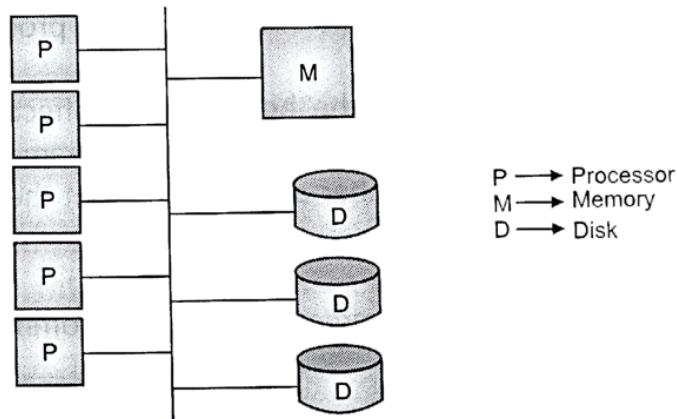
Characteristics of Shared Nothing (2/2)

- **Data Distribution:** Data is partitioned and each node is responsible for a specific subset of the data.
- **Parallel Processing:** Multiple nodes can perform operations concurrently on different partitions of the data, leading to significant performance improvements for large-scale tasks.



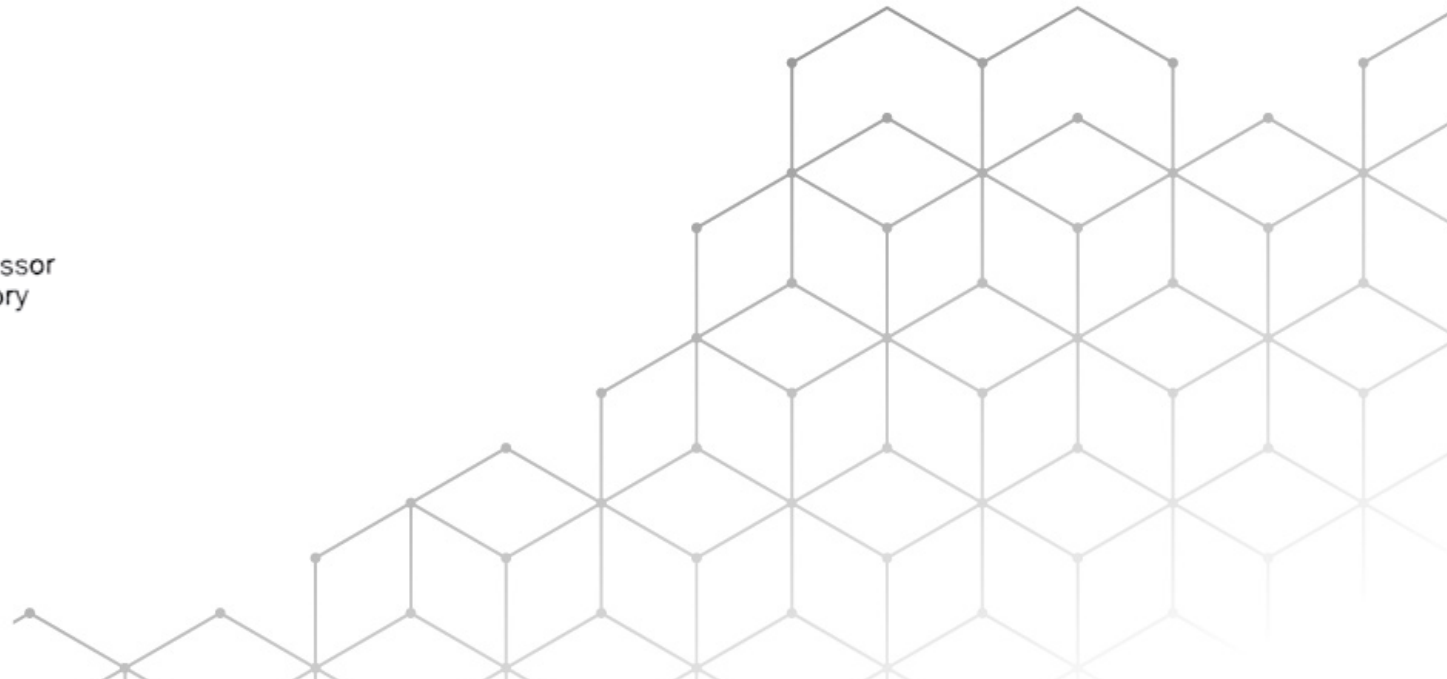
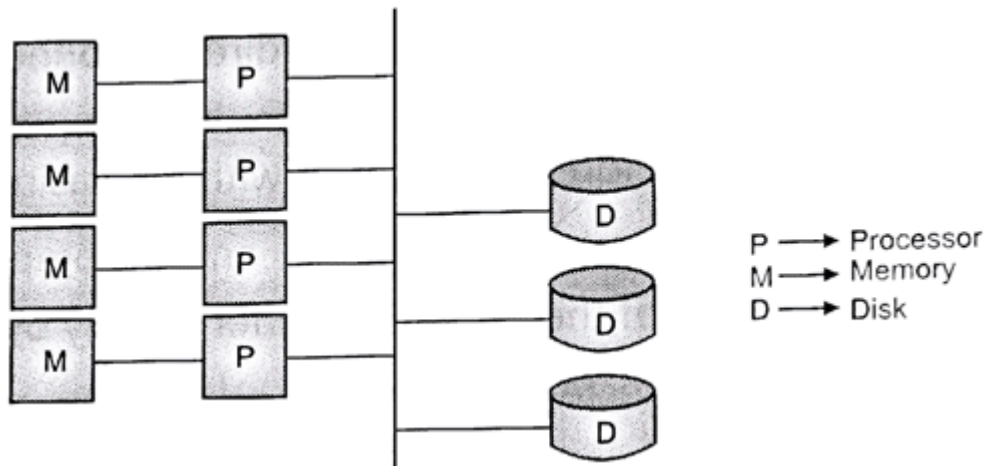
Architecture of Shared Memory

- Parallel processors share a common memory (one large memory).
- This architecture relies on extremely efficient communication between processors. No software interconnection.
- The main limitation is that it is not scalable beyond 32 or 64 CPUs.



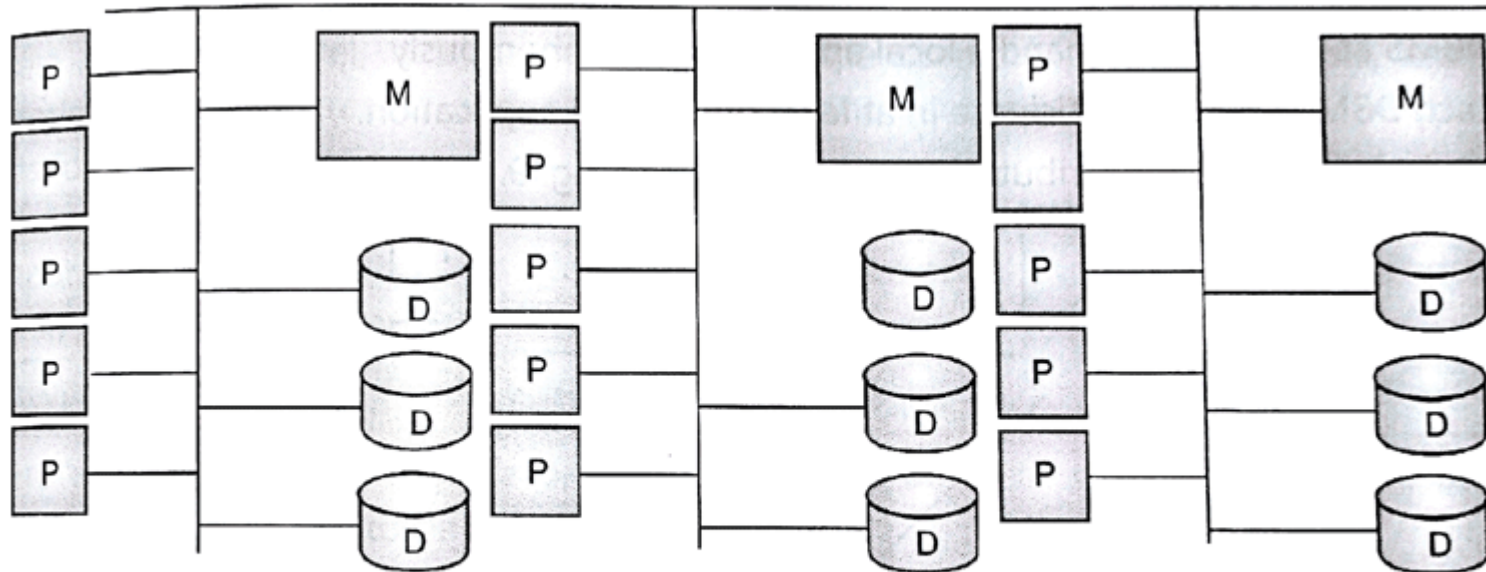
Architecture of Shared Disk

- Parallel processors share a common disk.
- This architecture can scale to a somewhat larger number of CPUs but communication between processors is slower and the bottleneck becomes the interconnection to the disk subsystem.



Hybrid (Hierarchical) Architecture

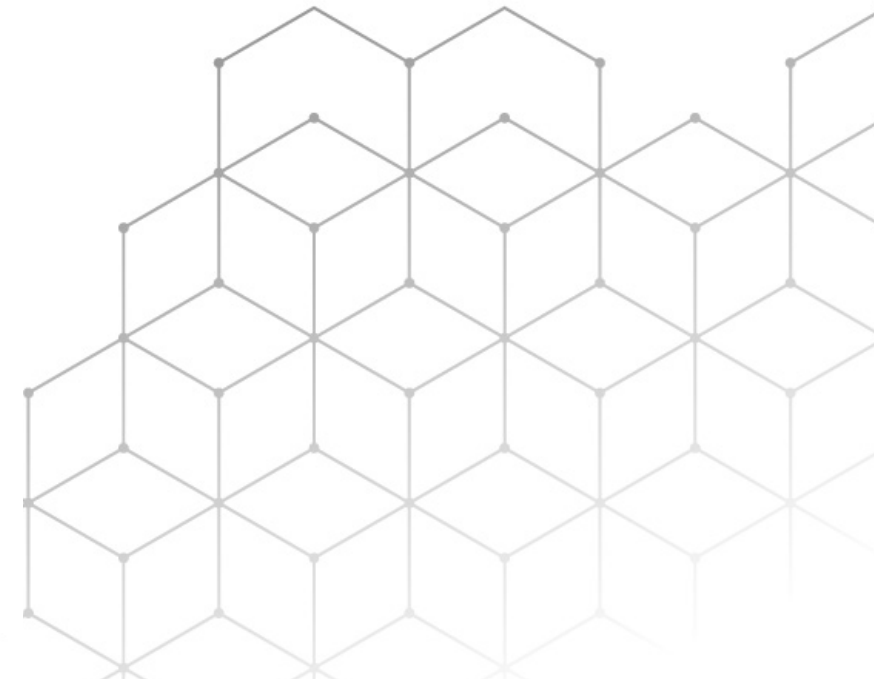
- Combines characteristics of shared-memory, shared-disk, and shared-nothing architectures.



P → Processor

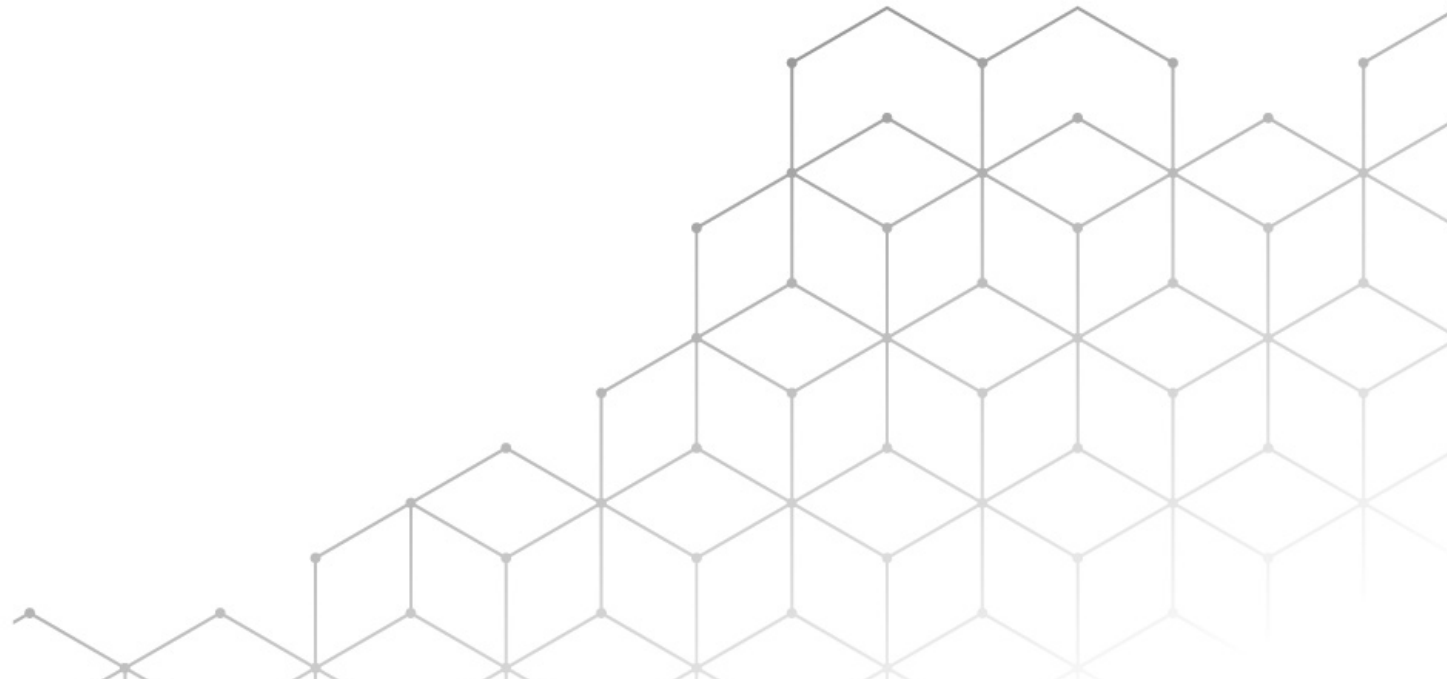
M → Memory

D → Disk



How about the Cloud?

- The cloud doesn't force a specific architecture — it enables you to choose the right one for your needs.
- Most modern, scalable systems in the cloud (like Netflix, Uber) use shared-nothing, because it's the best for scale and resilience.



Next Class

- Distributed Processing II: Big Data Analytics & MLlib



Questions and Thank you

